



BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN (FCC)

CARRERA: Lic. Ing. en Ciencias de la Computación

MATERIA: Programación Distribuida Aplicada

DOCENTE: Dr. Gustavo Emilio Mendoza Olguin

PROYECTO: Servicio distribuido DBaaS simplificado basado en
microservicios

ALUMNOS

Cristopher Damian Sánchez Santiago

Diego Yosef Tequipa Tepox

MATRICULAS

202253420

202254974

ÍNDICE

Introducción y Arquitectura General.....	3
Resumen Ejecutivo.....	3
Objetivos del Proyecto	3
Definición del Servicio (Database as a Service - DBaaS).....	4
Vista General de la Arquitectura de Microservicios	8
Abstracción y Capa de Datos Lógica.....	9
El Modelo de Datos Simplificado (SQL-like vs NoSQL-like).....	9
Implementación de Almacenamiento Interno (Estructuras tipo Colección)	17
Transparencia de Almacenamiento para el Usuario Final.....	18
Definición de Operaciones CRUD y Consultas de Agregación soportadas	18
Seguridad, Autenticación y Control de Acceso	24
Mecanismo de Autenticación Centralizada	24
Seguridad Local mediante JSON Web Tokens (JWT)	24
Control de Acceso Basado en Roles (RBAC)	25
Diseño de Interacciones y Flujos del Sistema	28
Diagrama de Secuencias de Operaciones Principales	28
Flujo de Mensajería Asíncrona con RabbitMQ.....	29
Comunicación Inter-servicio y Contratos gRPC.....	32
Protocolo de Comunicación gRPC	32
Definición de Contratos de Servicio (Archivos .proto)	37
Serialización de Datos y Tipos de Mensajes gRPC.....	37
Documentación Detallada de Interfaces y Servicios.....	38
Catálogo de Servicios y Endpoints expuestos (API Gateway)	38
Documentación de Colas y Formato de Mensajes (RabbitMQ)	44
Infraestructura de Red y Despliegue Físico	45
Topología de Red de los Nodos Distribuidos.....	45
Diagrama de Despliegue UML.....	48
Simulación de Escalabilidad y Clusterización (Filosofía de paso	48
Conclusiones	50
Evaluación del Cumplimiento de Objetivos (Transparencia e	50
Limitaciones del Sistema DBaaS Desarrollado	50
Referencias	51

Introducción y Arquitectura General

Resumen Ejecutivo

El presente proyecto expone el diseño, estructuración teórica e implementación arquitectónica de un **Servicio Distribuido de Base de Datos como Servicio (DBaaS) Simplificado**.

Este sistema se ha concebido bajo el paradigma de **desacoplamiento absoluto**. Para lograrlo, se diseñó un ecosistema modular compuesto por microservicios especializados que interactúan mediante dos canales de comunicación de alto rendimiento: de forma síncrona a través de llamadas a procedimientos remotos de alto desempeño (**gRPC sobre HTTP/2**) para operaciones críticas de control y consulta, y de forma asíncrona mediante un bróker de mensajería avanzado (**RabbitMQ**) enfocado en la absorción de carga, auditoría de logs y tareas secundarias del ciclo de vida del dato.

La capa de almacenamiento rompe con la rigidez de los sistemas tradicionales al implementar de manera lógica estructuras híbridas capaces de procesar interfaces tanto relacionales (**SQL-like**) como orientadas a documentos (**NoSQL-like**). Además, el acceso a los recursos y la integridad de los datos lógicos se encuentran blindados mediante una capa de seguridad local basada en componentes de autenticación criptográfica (**JWT**) y un estricto control de acceso basado en roles (**RBAC**). El resultado es una plataforma de software distribuida, transparente, de baja latencia y altamente escalable, lista para ser mapeada sobre infraestructura de red de alta disponibilidad.

Objetivos del Proyecto

Objetivo general

Diseñar, implementar y documentar la arquitectura de un servicio distribuido DBaaS simplificado basado en microservicios, que unifique interfaces de consulta relacionales y no relacionales, garantizando la transparencia de almacenamiento y la seguridad de los accesos mediante protocolos eficientes de comunicación y autenticación local.

Objetivos Específicos

- **Modularizar el sistema:** Fragmentar las responsabilidades operacionales en microservicios independientes (Autenticación, Gestión de Datos, API Gateway y Consultas) para eliminar puntos únicos de fallo y facilitar el escalamiento horizontal.
- **Abstraer la persistencia:** Desarrollar una interfaz de almacenamiento transparente para el usuario final, permitiendo que la manipulación interna de tablas o colecciones en memoria sea independiente de las peticiones del cliente.
- **Optimizar la comunicación Inter servicio:** Implementar contratos de servicio estrictos utilizando búferes de protocolo (**Protocol Buffers**) y **gRPC** para reducir la latencia de red y el uso de ancho de banda en transferencias críticas.
- **Asegurar el entorno:** Integrar un mecanismo de seguridad local basado en tokens de un solo uso (**JWT**) que transporte los roles del usuario para validar de manera inmediata los permisos sobre las operaciones CRUD.
- **Garantizar resiliencia asíncrona:** Diseñar una topología de mensajería con **RabbitMQ** para desacoplar las tareas pesadas de procesamiento de datos y asegurar que el sistema no pierda información ante picos de tráfico.

Definición del Servicio (Database as a Service - DBaaS)

DBaaS es la abreviatura de “Database as a Service” y describe la posibilidad de obtener uno o más **sistemas de bases de datos en la nube** de un proveedor de servicios correspondiente.

Es un servicio de **cloud computing** que permite a los usuarios acceder y utilizar el software de bases de datos sin necesidad de comprar y configurar hardware, instalar software o gestionar el sistema ellos mismos.

DBaaS: Database as a Service significa que las empresas que necesitan bases de datos relacionales o no relacionales para su trabajo diario ya no tienen por qué gestionarlos con sus propios trabajadores ni almacenarlos en sus propias infraestructuras informáticas, sino que pueden hacerlo mediante una cloud. Una conexión a Internet asegurada garantiza que todos los trabajadores y programas siempre tengan acceso a toda información relevante.

Beneficios

- **Ahorro de costes:** Con DBaaS, su organización paga un cargo periódico predecible basado en los recursos que consume; no es necesario comprar más capacidad para tener a mano hipotéticas necesidades futuras.
- **Escalabilidad aumento y reducción:** Puede aprovisionar rápida y fácilmente capacidad adicional de almacenamiento e informática en tiempo de ejecución si la necesita, y puede reducir su clúster de bases de datos durante las horas de uso no pico para ahorrar costes.
- **Gestión más sencilla y menos costosa:** Con DBaaS, el proveedor de servicios en la nube gestiona todo (aunque puede optar por gestionar ciertos aspectos usted mismo si lo desea). DBaaS aligera la carga administrativa de su personal de TI existente y lo libera para trabajar en aplicaciones e innovación.
- **Desarrollo rápido y tiempo de comercialización más rápido:** Con DBaaS, los desarrolladores pueden aprovechar las capacidades de la base de datos y crear y configurar una base de datos que esté lista para integrarse con su aplicación en cuestión de minutos.
- **Seguridad de datos y aplicaciones:** Los proveedores de bases de datos en la nube suelen ofrecer seguridad de nivel empresarial, incluidas características como el cifrado predeterminado de datos en reposo y en tránsito y controles de gestión de identidades y accesos integrados. Algunos también cumplen con estándares específicos de cumplimiento normativo.
- **Riesgo reducido:** Las ofertas DBaaS de los principales proveedores de servicios en la nube suelen incluir un acuerdo de nivel de servicio (SLA) que garantiza un determinado tiempo de actividad. En el improbable caso de que su proveedor no cumpla los requisitos estipulados en el SLA, se le compensará por cualquier tiempo de inactividad adicional que experimente.
- **Calidad del software:** Los principales proveedores de servicios en la nube ofrecen una amplia variedad de opciones de DBaaS altamente configurables, cada una preseleccionada por su calidad, para que no tenga que preocuparse por navegar entre cientos de bases de datos diferentes.

Ventajas	Inconvenientes
Costes de personal y tecnología más bajos	Los datos están fuera de la empresa
Menor esfuerzo administrativo	Los centros de datos pueden no estar accesibles provisionalmente
Informes exhaustivos	Las normas de protección de datos y las directrices de cumplimiento dependen del lugar de los centros de datos

Tabla 1 Ventajas / Inconvenientes de las DBaaS

Los principales proveedores de servicios en la nube ofrecen una amplia gama de opciones de DBaaS, incluidos los sistemas de gestión de bases de datos relacionales (RDBM), así como bases de datos no relacionales o NoSQL, como almacenes de documentos y columnas.

Encontrar el proveedor de DBaaS adecuado para su empresa implica determinar qué tecnologías de bases de datos funcionarán mejor para su aplicación y luego, por supuesto, asegurarse de que su proveedor es compatible con esa tecnología.

¿Cómo funciona DBaaS?

El funcionamiento de Database as a Service es bastante sencillo: dependiendo del acuerdo alcanzado en el contrato de servicios, un proveedor de la nube se compromete a proporcionar espacio de almacenamiento para un determinado número de bases de datos y a permitir los accesos correspondientes. Es posible facturar con una **tarifa basada en el uso**. Otra alternativa es que las empresas se limiten a alquilar recursos de servidor al proveedor para implantar ellas mismas una base de datos.

Las empresas que utilizan DBaaS pueden **confiar** la instalación y puesta a punto de las bases de datos, así como la atención técnica y el mantenimiento de los sistemas **totalmente al proveedor**, pues esto forma parte del contrato de servicios. Además de la asistencia puramente técnica, muchos proveedores de DBaaS ofrecen también otras funciones de confort, como un **monitoreo exhaustivo** de las bases de datos o la realización **automática de copias de seguridad periódicas** de los datos almacenados para reducir todo lo posible la pérdida de datos en caso de que el sistema caiga.

¿Para qué usos es adecuado el DBaaS?

En pocas palabras, la Database as a Service es útil para todos aquellos que quieran **gestionar una base de datos**, pero que no quieren o no pueden proporcionar la infraestructura y el esfuerzo de personal para hacerlo por sí mismos. El **enfoque DBaaS también tiene sentido** para aquellos que a veces se preocupan por la seguridad de sus datos, ya que los proveedores cuentan con expertos en informática con experiencia en la computación en la nube.

Como elegir un DBaaS

¿Se adaptará mejor a mi aplicación un almacén de datos primario o auxiliar?

Los almacenes de datos primarios son aquellos que ofrecen modelos de datos flexibles, incluidas bases de datos relacionales y almacenes de datos basados en documentos.

Por lo general, admiten lenguajes de consulta de propósito general (como las diversas implementaciones de SQL) y herramientas de modelado de datos de propósito general.

Algunos ejemplos de almacenes de datos primarios son **MySQL, MongoDB y PostgreSQL**.

Los almacenes de datos auxiliares, por el contrario, tienden a realizar bien unas pocas tareas especializadas, pero no son herramientas sólidas de uso general.

Ejemplos de este tipo son Redis, etcd, Elasticsearch y JanusGraph.

¿La arquitectura subyacente de la base de datos se adapta bien a mis necesidades?

Es fundamental elegir un motor de base de datos que no solo se adapte a los requisitos actuales de la aplicación, sino que también pueda ampliarse para satisfacer las necesidades futuras.

¿Realiza bien las pruebas la base de datos?

Dado que es tan fácil (y asequible) empezar a crear una oferta de DBaaS, una parte clave del proceso de selección debe ser la creación y exploración de un prototipo.

Esto le permite evaluar los tiempos de respuesta cuando su aplicación envía solicitudes reales a la base de datos y observar su rendimiento cuando se enfrenta a la combinación de operaciones y la cantidad de tráfico que encontrará en su entorno de producción.

¿Qué más ofrece el proveedor de servicios en la nube?

También es importante comparar las ofertas holísticas de los proveedores, que van más allá de las características y funciones de la propia base de datos.

¿Qué proveedores de DBaaS existen?

Hay algún que otro proveedor de Database as a Service. Las ofertas suelen diferenciarse solo en detalles y en las categorías de precios. Estos son algunos de los proveedores:

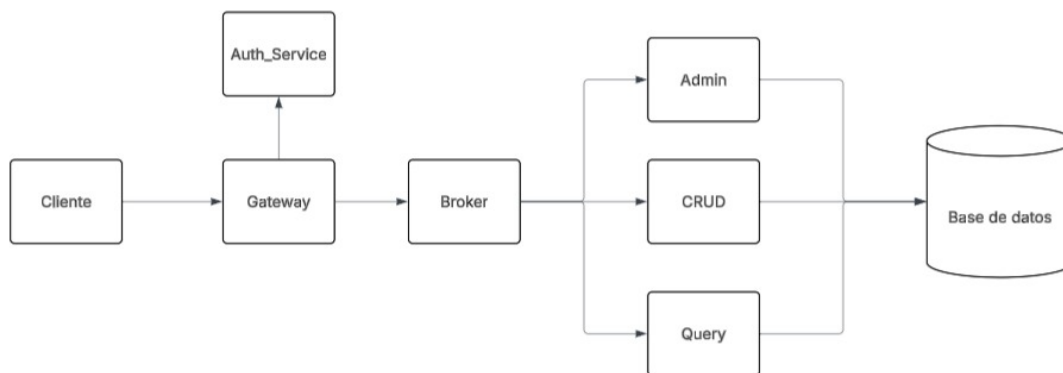
- **Amazon AWS**
- **Google**
- **Microsoft Azure**
- **MongoDB Atlas**
- **Oracle Cloud**

Vista General de la Arquitectura de Microservicios

El sistema se compone de una **topología descentralizada** donde cada microservicio ejecuta un rol específico dentro del ciclo de vida de la petición. Los seis microservicios implementados son:

- **MS1 — API Gateway (ms-gateway):** Único punto de entrada del sistema. Valida JWT, verifica permisos por rol y reenvía peticiones al broker.
- **MS2 — Translation Broker (ms-broker):** Recibe la petición cruda (SQL string o JSON), la parsea y la enruta al microservicio correcto mediante una Representación Interna (IR).
- **MS3 — Auth Service (ms-auth):** Gestiona registro, login y validación de tokens JWT. Persiste usuarios con contraseñas hasheadas con bcrypt en MySQL.

- **MS4 — Admin Service (ms-admin):** Crea y elimina bases de datos y tablas/colecciones en MySQL. Solo accesible para el rol admin.
- **MS5 — CRUD Service (ms-crud):** Ejecuta operaciones INSERT, SELECT, UPDATE y DELETE. Detecta automáticamente si la tabla es relacional o colección JSON.
- **MS6 — Query Service (ms-query):** Ejecuta agregaciones (COUNT, SUM, AVG, DISTINCT) y JOIN. Usa MPI para cómputo paralelo en las agregaciones escalares.



Abstracción y Capa de Datos Lógica

El Modelo de Datos Simplificado (SQL-like vs NoSQL-like)

SQL-like

El SQL (Structure Query Language), es un lenguaje de consulta estructurado establecido claramente como el lenguaje de alto nivel estándar para sistemas de base de datos relacionales. Los responsables de publicar este lenguaje como estándar, fueron precisamente los encargados de publicar estándar, la ANSI (Instituto Americano de Normalización) y la ISO (organismo Internacional de Normalización). Es por lo anterior que este lenguaje lo vas a encontrar en cualquiera de los DBMS relacionales que existen en la actualidad, por ejemplo, **ORACLE**, **SYBASES**, **SQL SERVER** por mencionar algunos.

El SQL agrupa tres tipos de sentencias con objetivos particulares, en los siguientes lenguajes:

- Lenguaje de Definición de Datos (**DDL, Data Definiton Language**)
- Lenguaje de Manipulación de Datos (**DML, Data Management Language**)
- Lenguaje de Control de Datos (**DCL, Data Control Language**)

Lenguaje de Definición de Datos (DDL, Data Definiton Language)

Grupo de sentencias del SQL que soportan la definición y declaración de los objetos de la base de datos. Objetos tales como: la base de datos misma (DATABASE), las tablas (TABLE), las Vistas (VIEW), los índices (INDEX), los procedimientos almacenados (PROCEDURE), los disparadores (TRIGGER), Reglas (RULE), Dominios (Domain) y Valores por defecto (DEFAULT).

CREATE,

ALTER y

DROP

Lenguaje de Manipulación de Datos (DML, Data Management Language)

Grupo de sentencias del SQL para manipular los datos que están almacenados en las bases de datos, a nivel de filas (tuplas) y/o columnas (atributos). Ya sea que se requiera que los datos sean modificados, eliminados, consultados o que se agregaren nuevas filas a las tablas de las bases de datos.

INSERT,

UPDATE,

DELETE y

SELECT

Lenguaje de Control de Datos (DCL, Data Control Language)

Grupo de sentencias del SQL para controlar las funciones de administración que realiza el DBMS, tales como la atomicidad y seguridad.

COMMIT TRANSACTION,

ROLLBACK TRANSACTION,

GRANT

REVOKE

Las BD relacionales (R-DBMS) se caracterizan por:

- Utilizar un modelo de datos simple (basado en tablas y relaciones entre tablas)
- Ofrecer herramientas para garantizar la integridad de datos y la consistencia de la información (ACID)
- Utilizar un lenguaje de interrogación estándar, simple y potente
- Proporcionar utilidades para asegurar el acceso, manipulación y la privacidad de los datos
- Ofrecer utilidades para la auditoría y recuperación de datos
- Garantizar la independencia del esquema lógico y físico

NoSQL-like

Es un término utilizado para describir un conjunto de bases de datos que se diferencian de las bases de datos relacionales (RDBMS) en los siguientes aspectos:

- Esquema prescindible, desnormalización, escalan horizontalmente y en sus premisas no está garantizar ACID

Hablar de bases de datos NoSQL es hablar de estructuras que nos permiten almacenar información en aquellas situaciones en las que las bases de datos relacionales generan ciertos problemas debido principalmente a problemas de escalabilidad y rendimiento de las bases de datos relacionales donde se dan cita miles de usuarios concurrentes y con millones de consultas diarias. Además de lo comentado anteriormente, las bases de datos NoSQL son sistemas de almacenamiento de información que **no cumplen con el esquema entidad-relación**. Tampoco utilizan una **estructura de datos en forma de tabla** donde se van almacenando los datos, sino que para el almacenamiento hacen uso de otros formatos como **clave-valor, mapeo de columnas o grafos**.

Principios

Tanto las bases de datos NoSQL como las relacionales son tipos de **Almacenamiento Estructurado**.

La principal diferencia radica en cómo guardan los datos (p.ej., el almacenamiento de una factura):

- En RDBMS separaríamos la información **en diferentes tablas** (cliente, factura, líneas_factura...) y luego el aplicativo, ejecutaría el **JOIN** y mapearía esta consulta SQL para mostrárselo al usuario.
- En NoSQL, simplemente se guarda la factura como **una unidad, sin separar los datos**.
- El control transaccional ACID no es importante.
- Los JOINS tampoco lo son. En especial los complejos y distribuidos. Se persigue la desnormalización.
- Algunos elementos relacionales son necesarios y aconsejables: claves (keys).
- Gran capacidad de escalabilidad y de replicación en múltiples servidores.

Su uso es adecuado para aquéllas que:

- Manejan volúmenes ingentes de datos
- Tienen una frecuencia alta de accesos de lectura y escritura
- Con cambios frecuentes en los esquemas de datos
- Y que no requieren consistencia ACID

Casos de aplicación:

- Servicios Web2.0 (redes sociales, blogs, etc.)
- Aplicaciones IoT
- Almacenamiento de perfiles sociales
- Juegos sociales
- Gestión de contenidos

En NoSQL, los datos se recuperan, en general, de forma más rápida que en RDBMS, sin embargo, las consultas que se puede realizar son más limitadas. La complejidad se traslada a la aplicación. NoSQL no es adecuado para aplicaciones que generen informes con consultas complejas (necesidad de JOINS), aunque tienen buena conexión con entornos como MapReduce que permite paralelizar operaciones complejas como agregaciones, filtros, etc..

Principales diferencias con las bases de datos SQL

- **No utilizan SQL como lenguaje de consultas.** La mayoría de las bases de datos NoSQL evitan utilizar este tipo de lenguaje o lo utilizan como un lenguaje de apoyo. Por poner algunos ejemplos, Cassandra utiliza el lenguaje CQL, MongoDB utiliza JSON o BigTable hace uso de GQL.
- **No utilizan estructuras fijas como tablas para el almacenamiento de los datos.** Permiten hacer uso de otros tipos de modelos de almacenamiento de información como sistemas de clave–valor, objetos o grafos.
- **No suelen permitir operaciones JOIN.** Al disponer de un volumen de datos tan extremadamente grande suele resultar deseable evitar los JOIN. Esto se debe a que, cuando la operación no es la búsqueda de una clave, la sobrecarga puede llegar a ser muy costosa. Las soluciones más directas consisten en desnormalizar los datos, o bien realizar el JOIN mediante software, en la capa de aplicación.
- **Arquitectura distribuida.** Las bases de datos relacionales suelen estar centralizadas en una única máquina o bien en una estructura máster–esclavo, sin embargo en los casos NoSQL la información puede estar compartida en varias máquinas mediante mecanismos de tablas Hash distribuidas.

Tipos de bases de datos NoSQL

Dependiendo de la forma en la que almacenen la información, nos podemos encontrar varios tipos distintos de bases de datos NoSQL.

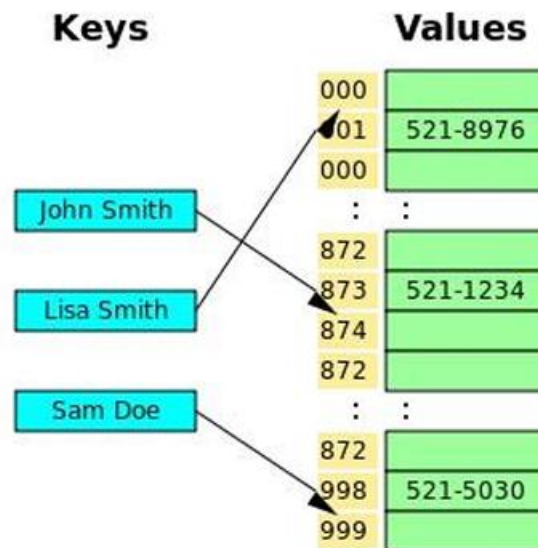


Ilustración 1 Bases de datos clave – valor

Son el modelo de base de datos NoSQL más popular, además de ser la más sencilla en cuanto a funcionalidad. En este tipo de sistema, cada elemento está identificado por una llave única, lo que permite la recuperación de la información de forma muy rápida, información que habitualmente está almacenada como un objeto binario (BLOB). Se caracterizan por ser muy eficientes tanto para las lecturas como para las escrituras.

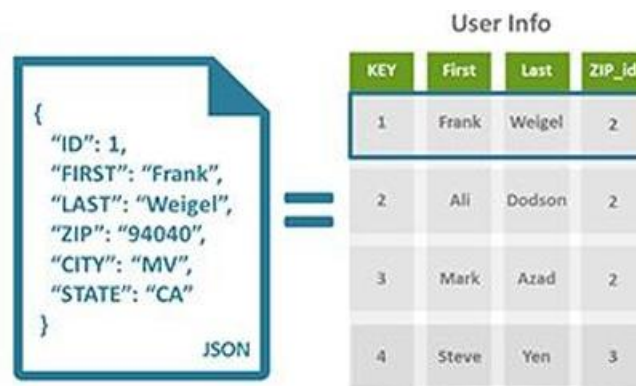


Ilustración 2 Bases de datos documentales

Este tipo almacena la información como un documento, generalmente utilizando para ello una estructura simple como JSON o XML y donde se utiliza una clave única para cada registro. Este tipo de implementación permite, además de realizar búsquedas por clave-valor, realizar consultas más avanzadas sobre el contenido del documento. Son las bases de datos NoSQL más versátiles. Se pueden utilizar en gran cantidad de proyectos, incluyendo muchos que tradicionalmente funcionarían sobre bases de datos relacionales.

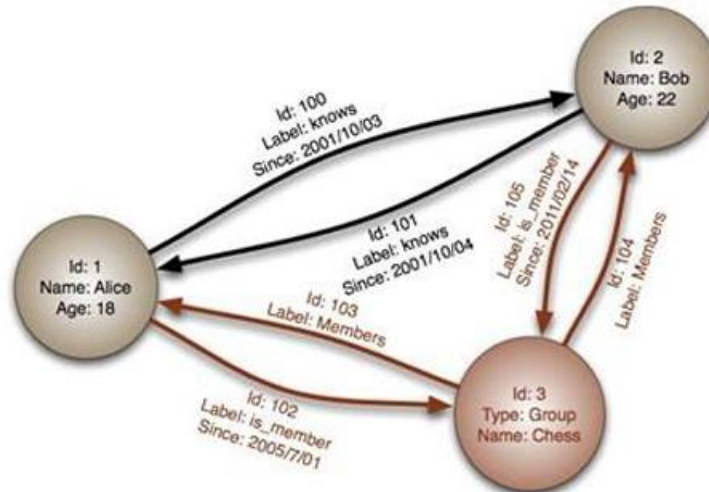


Ilustración 3 Bases de datos en grafo

En este tipo de bases de datos, la información se representa como nodos de un grafo y sus relaciones con las aristas del mismo, de manera que se puede hacer uso de la teoría de grafos para recorrerla. Para sacar el máximo rendimiento a este tipo de bases de datos, su estructura debe estar totalmente normalizada, de forma que cada tabla tenga una sola columna y cada relación dos.

En las bases de datos orientadas a objetos, la información se representa mediante objetos, de la misma forma que son representados en los lenguajes de programación orientada a objetos (POO) como ocurre en JAVA, C# o Visual Basic .NET.

Interfaz SQL-like (Orientada a Esquemas y Relaciones)

La interacción relacional está gestionada en el cliente por el componente **interfaces/sql_interface.py**. Este módulo expone una interfaz declarativa emulando el comportamiento de un SGBD relacional tradicional.

- **Comportamiento Lógico:** El usuario define estructuras rígidas compuestas por tablas, columnas y tipos de datos explícitos.
- **Procesamiento de Consultas:** Cuando el cliente envía una instrucción de este tipo, el archivo **ms-broker/parsers/sql_parser.py** intercepta la cadena de texto y la convierte en una **Representación Interna (IR)** abstracta.
- **Traducción y Ejecución:** Esta IR es enviada al microservicio de operaciones CRUD (**ms-crud**), donde el módulo **ir_to_sql.py** traduce la petición a

consultas SQL nativas para ser ejecutadas finalmente por el componente **executor.py**.

Interfaz NoSQL-like (Orientada a Colecciones y Documentos Flexibles)

La interacción no relacional está asistida por el componente `interfaces/nosql_interface.py` en el cliente, diseñada para desarrolladores que requieren un almacenamiento ágil y polimórfico en formato de objetos de tipo JSON.

- **Comportamiento Lógico:** El usuario interactúa con el sistema mediante el concepto de **Colecciones**. No existe una definición previa de esquemas o columnas; cada registro (documento) inyectado posee su propia estructura y puede contener arreglos o diccionarios embebidos de manera dinámica.
- **Procesamiento de Consultas:** Las peticiones orientadas a documentos evitan el procesamiento relacional pesado. El componente **ms-broker/parsers/nosql_parser.py** parsea directamente los objetos lógicos y las instrucciones basadas en métodos (como operaciones de agregación o búsquedas por llaves) para enviarlas al motor de ejecución de manera directa y optimizada.

Dimensión lógica	SQL-like (<code>sql_interface.py</code>)	NoSQL-like (<code>nosql_interface.py</code>)
Abstracción Principal	Tablas lógicas estructuradas	Colecciones de documentos flexibles
Componente analizador	<code>parsers/sql_parser.py</code>	<code>parsers/nosql_parser.py</code>
Formato de Tránsito	Representación Interna (IR) basada en Esquemas	Estructuras de Clave – Valor / JSON Dinámico
Destino de Ejecución	Traducción a MySQL en <code>ms-crud/ir_to_sql.py</code>	Pipeline de agregación directa/MPI en <code>ms-query</code> .

Flexibilidad	Estricta (Tipos de datos y campos validados).	Alta (Polimorfismo por documento).
--------------	---	------------------------------------

Tabla 2 Solución de forma transparente a ambos paradigmas lógicos

Implementación de Almacenamiento Interno (Estructuras tipo Colección)

El sistema utiliza MySQL como motor de almacenamiento principal. Las colecciones se implementan como tablas MySQL con una columna `_id` (VARCHAR UUID) y una columna `_data` (JSON), mientras que las tablas relacionales usan columnas explícitas definidas por el usuario.

La distinción entre ambos modos se detecta en tiempo de ejecución consultando `SHOW COLUMNS FROM tabla LIKE '_data'`. Si existe esa columna, el sistema opera en modo colección; de lo contrario, opera en modo tabla.

Estructuras de datos usadas en memoria (Python):

- **dict** — representa registros individuales y resultados de queries. También se usa como tabla de despacho para mapear operaciones a funciones (parsers, handlers).
- **list** — almacena conjuntos de resultados (documents, rows) retornados desde MySQL.
- **set** — usado en `role_guard.py` para definir y verificar permisos por rol en $O(1)$.

Representación interna (IR): todas las operaciones, sin importar si vienen de la interfaz SQL o NoSQL, se homogenizan en un dict Python con campos `operation`, `database`, `table`, `fields`, `filter`, `data` y `options` antes de ejecutarse contra MySQL.

Transparencia de Almacenamiento para el Usuario Final

El principio de transparencia de almacenamiento garantiza que el cliente no necesita conocer ni distinguir cómo se persisten sus datos internamente. Este principio es uno de los pilares de los sistemas distribuidos modernos (Tanenbaum & Van Steen, 2007).

En el sistema implementado, la transparencia se logra mediante tres mecanismos:

1. **La IR unificada:** tanto `sql_parser.py` como `nosql_parser.py` producen exactamente el mismo formato de dict IR. El broker, el servicio CRUD y el servicio de consultas operan únicamente sobre esta IR, sin importar el origen.
2. **Detección automática de modo:** `executor.py` llama a `is_collection()` antes de traducir cualquier operación. El usuario nunca especifica si trabaja con una tabla relacional o una colección; el sistema lo detecta solo.
3. **API única al cliente:** el cliente siempre habla con un único punto de entrada (Gateway en puerto 50051). No existe visibilidad de los servicios internos, MySQL, ni RabbitMQ.

Definición de Operaciones CRUD y Consultas de Agregación soportadas

La manipulación de los datos consiste en la realización de operaciones de inserción, borrado, modificación y consulta de la información almacenada en la base de datos. La inserción y el borrado son el resultado de añadir nueva información a la ya que se encontraba almacenada o eliminarla de nuestra base de datos, tomando en cuenta las restricciones marcadas por el DDL y las relaciones entre la nueva información y la antigua.

La modificación nos permite alterar esta información, y la consulta nos permite el acceso a la información almacenada en la base de datos siguiendo criterios específicos. En general a las operaciones básicas de manipulación de datos que podemos realizar con SQL se les denomina operaciones CRUD (de Create, Read, Update and Delete, o sea, Crear, Leer, Actualizar y Borrar, sería CLAB en español, pero no se usa).

INSERTAR DATOS: SENTENCIA INSERT

La sentencia **INSERT INTO** se utiliza para insertar nuevos registros en una tabla.

Es posible escribir la sentencia **INSERT INTO** de dos formas:

1. Especificando los nombres de las columnas y los valores que se insertarán:

```
INSERT INTO nombre_tabla (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

Ilustración 4 Forma 1 Sentencia INSERT INTO

2. Si se agrega valores para todas las columnas de la tabla, no es necesario que se especifiquen los nombres de las columnas en la consulta SQL. Sin embargo, hay que asegurarse de que el orden de los valores esté en el mismo orden que las columnas de la tabla. Aquí, la sintaxis INSERT INTO sería la siguiente:

```
INSERT INTO nombre_tabla  
VALUES (value1, value2, value3, ...);
```

Ilustración 5 Forma 2 Sentencia INSERT INTO

ACTUALIZAR DATOS: SENTENCIA UPDATE

La sentencia **UPDATE** se utiliza para modificar los registros existentes en una tabla.

```
UPDATE nombre_tabla  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

Ilustración 6 Sentencia UPDATE

BORRAR DATOS: SENTENCIA DELETE

La sentencia **DELETE** se utiliza para eliminar los registros existentes en una tabla.

```
DELETE FROM nombre_tabla WHERE condition;
```

Ilustración 7 Sentencia DELETE

SENTENCIA TRUNCATE TABLE

Quita todas las filas de una tabla o las particiones especificadas de una tabla, sin registrar las eliminaciones individuales de filas. TRUNCATE TABLE es similar a la instrucción DELETE sin una cláusula WHERE; no obstante, TRUNCATE TABLE es más rápida y utiliza menos recursos de registros de transacciones y de sistema.

```
TRUNCATE TABLE nombre_tabla;
```

Ilustración 8 Sentencia TRUNCATE

CONSULTAR DATOS: SENTENCIA SELECT

La sentencia **SELECT** se utiliza para seleccionar datos de una base de datos. Los datos devueltos se almacenan en una tabla de resultados, denominada conjunto de resultados.

```
SELECT {*/campos} FROM nombre_tabla {WHERE condition};
```

Ilustración 9 Sentencia SELECT

Operación	SQL-like	NoSQL-like	Servicio destino
Insertar registro	INSERT INTO tabla (cols) VALUES (...)	{"op": "insert", "document": {...}}	ms-crud
Consultar registros	SELECT campos FROM tabla WHERE ...	{"op": "find", "filter": {...}}	ms-crud
Actualizar registros	UPDATE tabla SET col=val WHERE ...	{"op": "update", "filter": {}, "update": {}}	ms-crud
Eliminar registros	DELETE FROM tabla WHERE ...	{"op": "delete", "filter": {...}}	ms-crud
Crear base de datos	CREATE DATABASE nombre	{"op": "create_db", "db": "nombre"}	ms-admin
Eliminar base de datos	DROP DATABASE nombre	{"op": "drop_db", "db": "nombre"}	ms-admin
Crear tabla	CREATE TABLE nombre (cols)	{"op": "create_table", "mode": "table"}	ms-admin
Crear colección	CREATE COLLECTION nombre	{"op": "create_table", "mode": "collection"}	ms-admin

Tabla 3 Operaciones CRUD soportadas

En el mundo de SQL, las funciones agregadas son herramientas esenciales para resumir y analizar datos de forma eficaz. Tienen una capacidad única para destilar grandes conjuntos de datos en información significativa, para facilitar el análisis estadístico y para simplificar estructuras de datos complejas.

CONTAR()

El objetivo de esta función es contar el número de filas de una tabla o el número de valores no nulos de una columna.

```
SELECT COUNT(*) as total_products
FROM products;
```

Ilustración 10 Consulta COUNT

SUMA()

La función **SUM()** devuelve el total de una columna numérica. Suele utilizarse cuando se necesita hallar el total de valores como ingresos por ventas, cantidades o gastos.

```
SELECT SUM(sales_amount) as total_revenue
FROM sales_data;
```

Ilustración 11 Consulta SUM

AVG()

Resulta útil cuando se busca el precio medio, la valoración, las unidades vendidas, etc.

```
SELECT AVG(session_duration) as average_session_duration
FROM user_sessions;
```

Ilustración 12 Consulta AVG

DISTINCT()

Resulta útil cuando se busca eliminar registros duplicados de una consulta y obtener una lista de valores únicos a partir de una columna o atributo específico dentro de una colección.

```
SELECT DISTINCT sale_date
FROM customers;
```

Ilustración 13 Consulta DISTINCT

INNER JOIN()

Resulta útil cuando se busca combinar filas de dos o más tablas de forma lógica, basándose en una columna común o llave primaria/foránea compartida, para consolidar información relacionada en un solo conjunto de resultados.

```
SELECT columnas
FROM tabla1
INNER JOIN tabla2
ON tabla1.columna = tabla2.columna;
```

Ilustración 14 Consulta INNER JOIN

Operación	SQL-like	NoSQL-like	Motor de cómputo
COUNT	SELECT COUNT(*) FROM tabla	{"op": "count", "field": "*"}	MPI (paralelo)
SUM	SELECT SUM(campo) FROM tabla	{"op": "sum", "field": "campo"}	MPI (paralelo)
AVG	SELECT AVG(campo) FROM tabla	{"op": "avg", "field": "campo"}	MPI (paralelo)
DISTINCT	SELECT DISTINCT campo FROM tabla	{"op": "distinct", "field": "campo"}	MySQL directo
INNER JOIN	SELECT * FROM a INNER JOIN b ON a.id=b.fk	{"op": "join", "join_collection": "b", "on": "..."} {"op": "join", "join_collection": "b", "on": "..."}	ms-query / Python

Tabla 4 Consultas de agregación soportadas

Seguridad, Autenticación y Control de Acceso

Mecanismo de Autenticación Centralizada

El sistema implementa autenticación centralizada a través del microservicio **ms-auth**. Este servicio es el único componente que conoce y gestiona las credenciales de los usuarios. Ningún otro microservicio almacena o valida contraseñas directamente; todos delegan esta responsabilidad al Auth Service a través del API Gateway.

El flujo de autenticación sigue el patrón Token-Based Authentication, ampliamente documentado como práctica recomendada para sistemas distribuidos (Jones et al., RFC 7519, 2015). El proceso es:

1. El cliente envía credenciales (**usuario y contraseña**) al **Gateway** vía **gRPC**.
2. El **Gateway** reenvía las credenciales al **Auth Service (ms-auth)**.
3. El **Auth Service** consulta la tabla **users** en **MySQL** y verifica la contraseña con **bcrypt**.
4. Si son válidas, genera un **JWT** firmado con clave secreta (**HS256**) y lo retorna al cliente.
5. El cliente incluye este **JWT** en todas las peticiones posteriores.
6. El **Gateway** valida el **JWT** en cada petición llamando a **ValidateToken** en **ms-auth**.

Las contraseñas se almacenan con **bcrypt** (factor de coste adaptativo), que incorpora sal aleatoria en cada hash, haciendo inviable el uso de tablas rainbow. Esto cumple con las recomendaciones del NIST SP 800-63B para almacenamiento seguro de credenciales (NIST, 2017).

Seguridad Local mediante JSON Web Tokens (JWT)

Un **JSON Web Token (JWT)** es un estándar abierto (RFC 7519) que define un formato compacto y autocontenido para transmitir información entre partes de manera segura como un objeto **JSON firmado** (Jones et al., 2015).

Estructura del JWT implementado:

El token consta de tres partes separadas por puntos (**header.payload.signature**):

- **Header: algoritmo de firma** {"alg": "HS256", "typ": "JWT"}
- **Payload: datos del usuario** {"user_id": "uuid", "username": "...", "role": "admin|write|read", "iat": timestamp, "exp": timestamp+24h}
- **Signature: HMAC-SHA256**(base64(header) + "." + base64(payload), JWT_SECRET)

La firma **HMAC-SHA256** garantiza que cualquier modificación al token sea **detectada**, ya que el **Gateway re-verifica** la firma en cada petición llamando a **ValidateToken**. El token expira en 24 horas (**configurable via JWT_EXP_HOURS**). Los errores manejados son: token expirado (**ExpiredSignatureError**) y token inválido (**InvalidTokenError**).

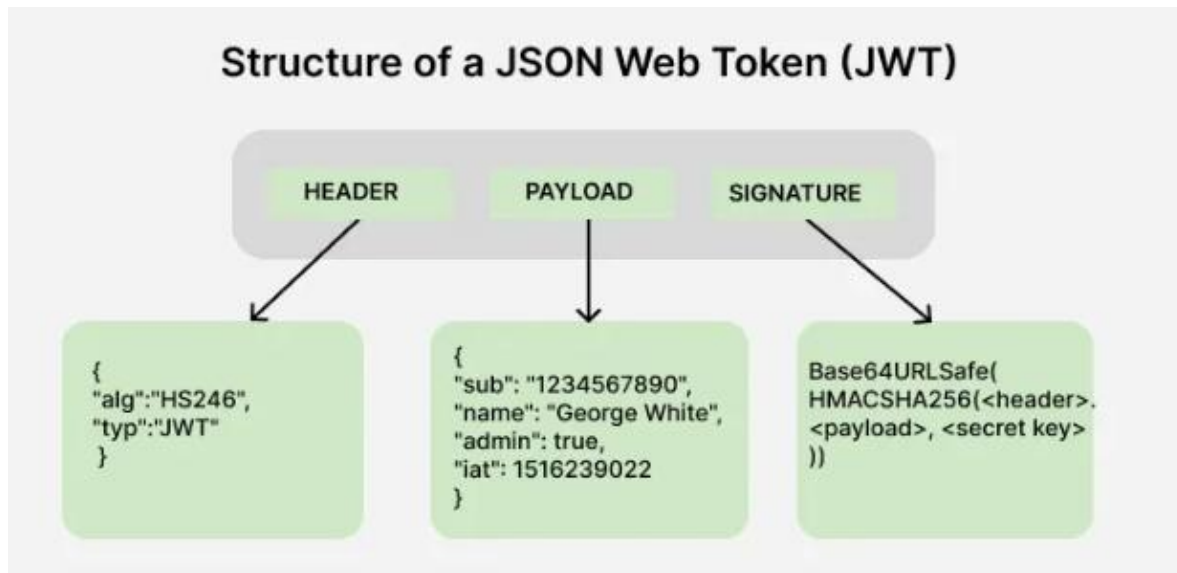


Ilustración 15 Estructura JSON (JWT)

Ventajas del enfoque stateless:

- No requiere almacenamiento de sesiones en el servidor.
- Cada microservicio puede verificar el token de manera autónoma sin consultar una base de datos central.
- Escalabilidad horizontal sin estado compartido entre instancias.

Control de Acceso Basado en Roles (RBAC)

El Control de acceso basado en roles, o RBAC como son sus siglas, es un enfoque para restringir el acceso al sistema a usuarios no autorizados.

Tiene como objetivo asegurar la confidencialidad, integridad y disponibilidad de la información, puesto que, basándose en el principio de privilegio mínimo, limita el acceso de los usuarios y las acciones que pueden llevar a cabo. De esta forma, se reduce el riesgo de sufrir violaciones de seguridad cuando la cuenta de un usuario se ve comprometida.



Ilustración 16 Control de Acceso RBAC

Dentro de una organización, se crean roles para diversas funciones. Los permisos para realizar ciertas operaciones se asignan a roles específicos. A los usuarios se les asignan roles particulares y, a través de esas asignaciones de roles, adquieren los permisos para realizar funciones específicas del sistema informático.

Dado que a los usuarios no se les asignan permisos directamente, ya que solo los adquieren a través de su rol (o roles), la administración de los derechos de usuario individuales se convierte en una cuestión de simplemente asignar los roles apropiados a la cuenta del usuario; esto simplifica las operaciones comunes, como agregar un usuario o cambiar el departamento de un usuario.

El sistema implementa Role-Based Access Control (RBAC), un modelo de control de acceso donde los permisos se asignan a roles, y los roles se asignan a usuarios. Este modelo es el más ampliamente adoptado en sistemas empresariales (Sandhu et al., 1996).

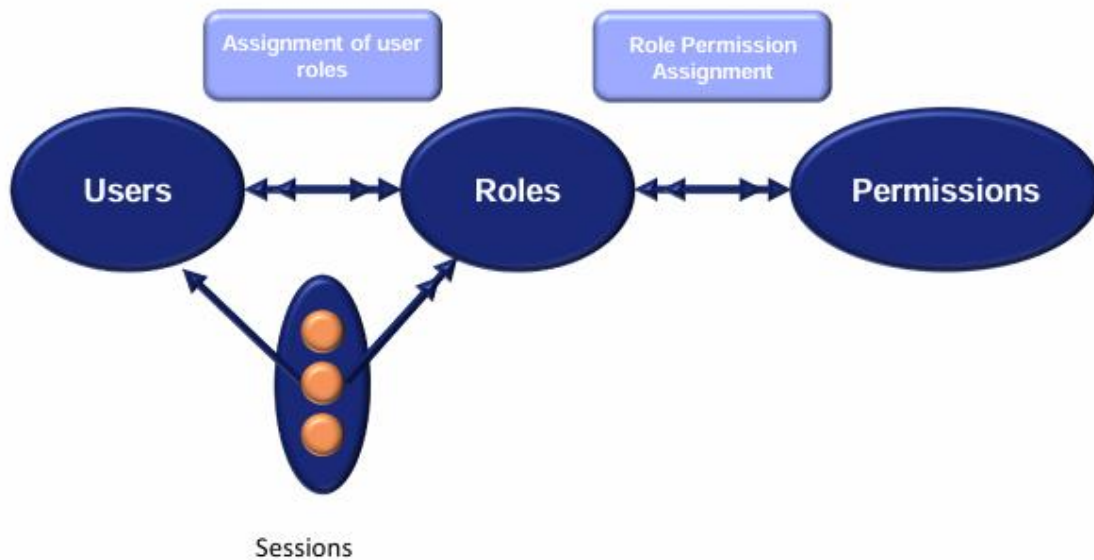


Ilustración 17 Diagrama RBAC

Como se ve en el diagrama, un usuario puede ser miembro de muchos roles y un rol puede tener muchos usuarios. De manera similar, un rol puede tener muchos permisos y el mismo permiso se puede asignar a muchos roles. La clave de RBAC radica en estas dos relaciones. Finalmente, una sesión puede estar compuesta por un usuario y muchos roles.

Roles definidos en el sistema:

- **admin:** acceso completo. Puede crear/eliminar bases de datos y tablas, además de todas las operaciones de escritura y lectura.
- **write:** puede insertar, actualizar y eliminar registros, además de todas las operaciones de lectura y agregación.
- **read:** acceso de solo lectura. Puede ejecutar SELECT, COUNT, SUM, AVG, DISTINCT y JOIN.

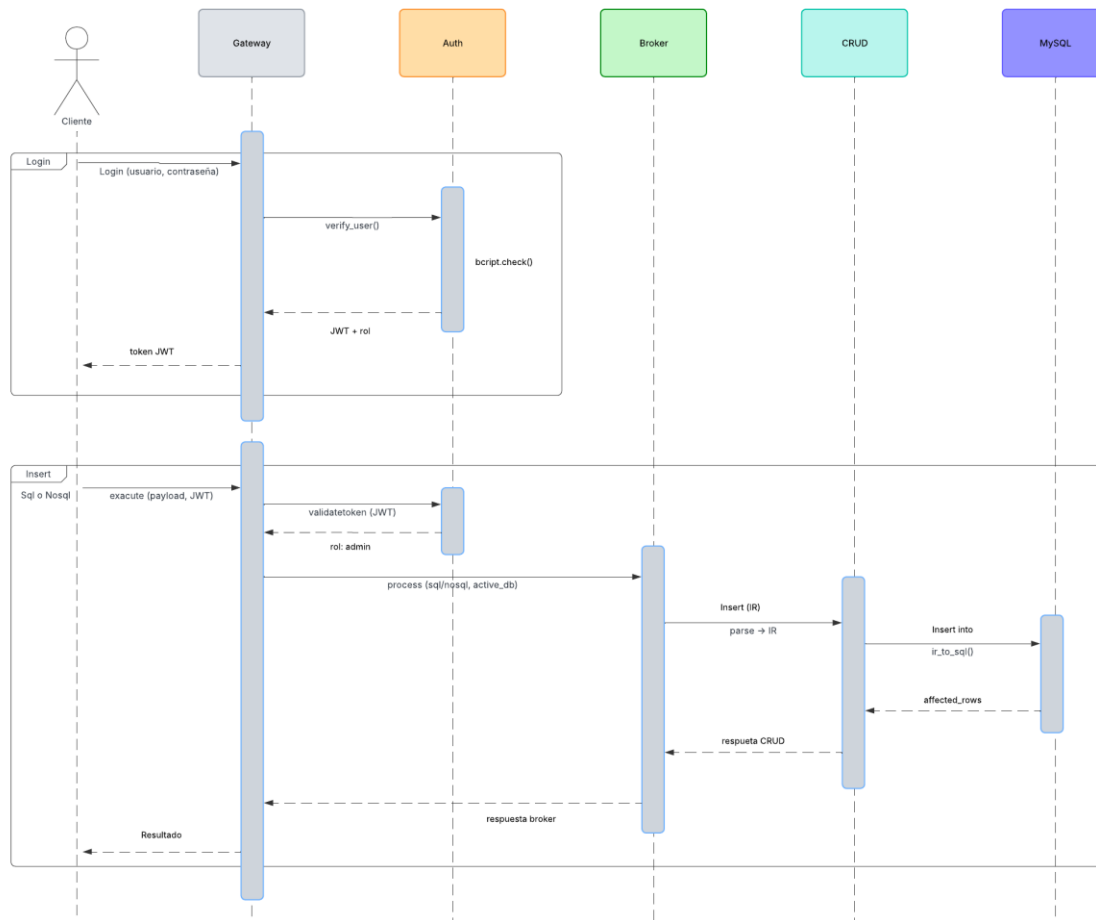
Operación	admin	write	read
CREATE DATABASE / DROP DATABASE	✓	X	X
CREATE TABLE / DROP TABLE	✓	X	X
SHOW DATABASES / SHOW TABLES	✓	✓	✓
INSERT	✓	✓	X
UPDATE	✓	✓	X
DELETE	✓	✓	X
SELECT (find)	✓	✓	✓
COUNT / SUM / AVG	✓	✓	✓
DISTINCT / JOIN	✓	✓	✓

Tabla 5 Matriz de permisos RBAC por operación

El control de acceso se aplica en dos capas: primero en el Gateway (role_guard.py) que verifica el rol extraído del JWT antes de reenviar la petición, y nuevamente en cada microservicio destino (admin_server.py, crud_server.py) como segunda línea de defensa. Este patrón de validación en múltiples capas es una práctica recomendada de seguridad en profundidad (defense in depth).

Diseño de Interacciones y Flujos del Sistema

Diagrama de Secuencias de Operaciones Principales



Flujo de Mensajería Asíncrona con RabbitMQ

RabbitMQ es un broker de mensajería de código abierto que implementa el protocolo AMQP (Advanced Message Queuing Protocol). Permite el desacoplamiento entre productores y consumidores de mensajes, garantizando la entrega incluso cuando el consumidor no está disponible temporalmente (RabbitMQ Documentation, 2024).

Es un software que actúa como Message Broker o lo que quiere decir lo mismo, el **lugar donde publicamos mensajes** cuando queremos tener comunicación asíncrona en nuestra aplicación.

También nos permite administrar colas, las cuales funcionan como las **colas de las colecciones de .NET**.

Las colas en RabbitMQ funcionan con la misma idea que las colas en la programación en general, **llega un elemento por un lado y sale por el otro**,

siempre de forma ordenada y **siempre 1 a 1**. Lo que quiere decir que llega un mensaje por un lado y sale un solo mensaje por el otro.

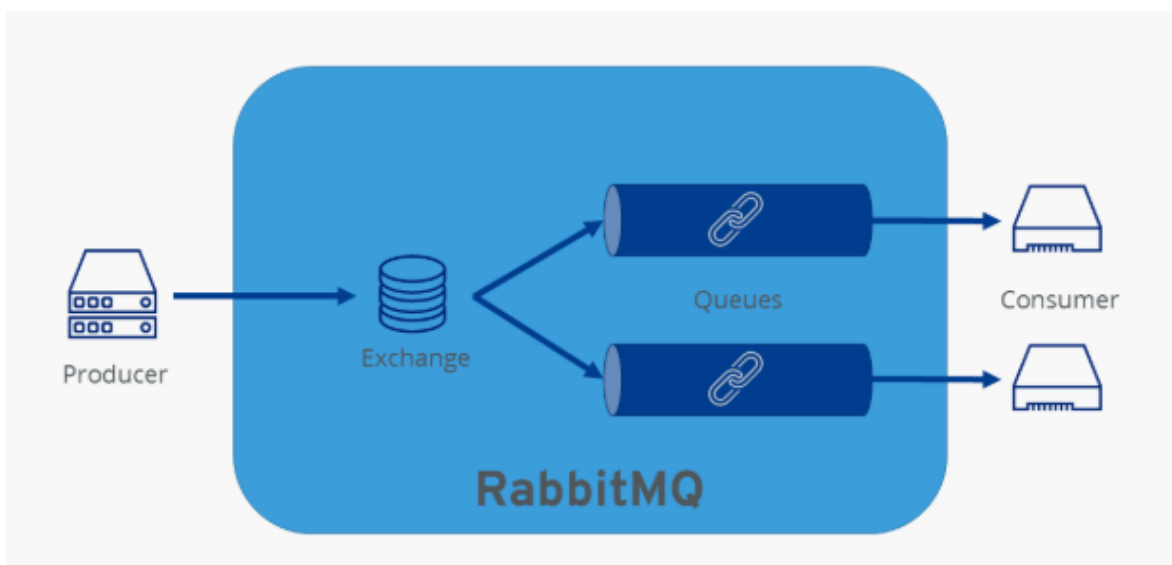


Ilustración 18 Procedimiento del paso de mensajes por medio de colas en RabbitMQ

Los exchanges, por el contrario, son algo más complicados; En resumidas cuentas, podríamos entender un exchange como el lugar donde debemos publicar los mensajes cuando queremos tener la posibilidad de **tener más de un consumidor**.

Los exchanges en RabbitMQ funcionan de una forma un poco peculiar, y es que **no almacenan la información**, únicamente la transfieren, eso quiere decir que si enviamos un mensaje a un exchange, y no tienen ningún consumidor, este **se pierde**.

Pero a la vez, **no podemos consumir de un exchange directamente en nuestra aplicación**, si no que únicamente podemos consumir de las colas.

Por lo tanto, **debemos enlazar los exchanges con las colas** correspondientes, a esta acción se le llama hacer un **binding**. Y juntos, Exchange, binding y colas, representan un **broker**.

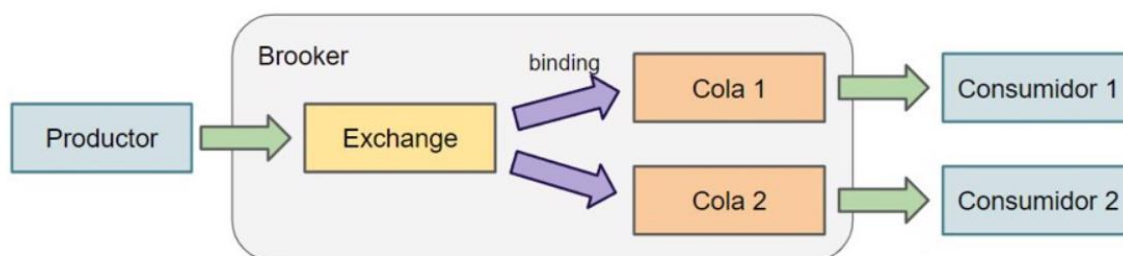


Ilustración 19 Procedimiento del Broker

La mensajería asíncrona es un componente crucial en la arquitectura de microservicios. Permite que los servicios se comuniquen entre sí sin necesidad de esperar una respuesta inmediata, lo que mejora la escalabilidad y la resiliencia del sistema.

Implementación en el sistema:

El **Gateway** actúa como productor: después de cada operación **Execute exitosa o fallida, publica** un evento de auditoría en la cola **dbaas.audit** de **RabbitMQ**. El proceso **ms-audit** actúa como **consumidor**: lee los mensajes de la cola y los persiste en la tabla **audit_log** de **MySQL**.

Características de la cola:

- **Cola durable:** sobrevive a reinicios de RabbitMQ. Los mensajes no se pierden ante caídas del broker.
- **Mensajes persistentes:** `delivery_mode=2` garantiza que los mensajes se escriban en disco.
- **Prefetch count=1:** el consumidor procesa un mensaje a la vez para no saturar MySQL.
- **ACK manual:** el mensaje se confirma (ACK) solo después de ser guardado en MySQL, evitando pérdidas.
- **NACK sin requeue:** mensajes corruptos son rechazados sin reencolar para evitar loops infinitos.

Payload del mensaje de auditoría (JSON):

```
{ "user_id": "uuid", "role": "admin|write|read", "operation": "find|insert|...", "interface":  
"sql|nosql", "success": true|false }
```

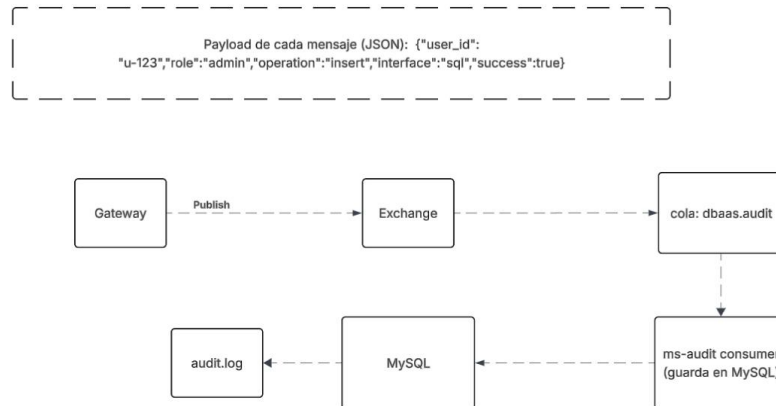


Ilustración 20 Paso de mensajes RabbitMQ

Estrategia de desacoplamiento:

Si el servicio **ms-audit** cae, el **Gateway** continúa operando normalmente. Los eventos de auditoría **se acumulan en la cola durable** y se procesan cuando **ms-audit** se recupera. Esto garantiza que las **operaciones críticas** del usuario nunca **se bloqueen por tareas secundarias de auditoría**.

Comunicación Inter-servicio y Contratos gRPC

Protocolo de Comunicación gRPC

gRPC (Google Remote Procedure Call) es un framework de comunicación de alto rendimiento desarrollado por Google, lanzado como proyecto de código abierto en 2015. Utiliza HTTP/2 como protocolo de transporte y Protocol Buffers (protobuf) como mecanismo de serialización, lo que lo hace significativamente más eficiente que REST/JSON en términos de tamaño de mensaje y latencia (Google, 2024).

RPC, o llamada a procedimiento remoto, es un modelo de comunicación para la interacción cliente/servidor que permite que las llamadas remotas aparezcan y funcionen como llamadas locales.

En una RPC, el cliente interactúa con una representación del servidor, que parece local pero, de hecho, es un intermediario. Este intermediario se conoce comúnmente como stub, que maneja los datos de serialización y desclasificación

(es decir, convertir los datos a un formato adecuado para la transmisión y convertir los resultados recibidos del servidor al formato original).

Sin embargo, muchos entornos modernos, especialmente aquellos que utilizan arquitecturas de microservicios, entornos políglotas y sistemas con altas cargas de datos, requieren una infraestructura más rápida y de alto rendimiento para conectar aplicaciones distribuidas.

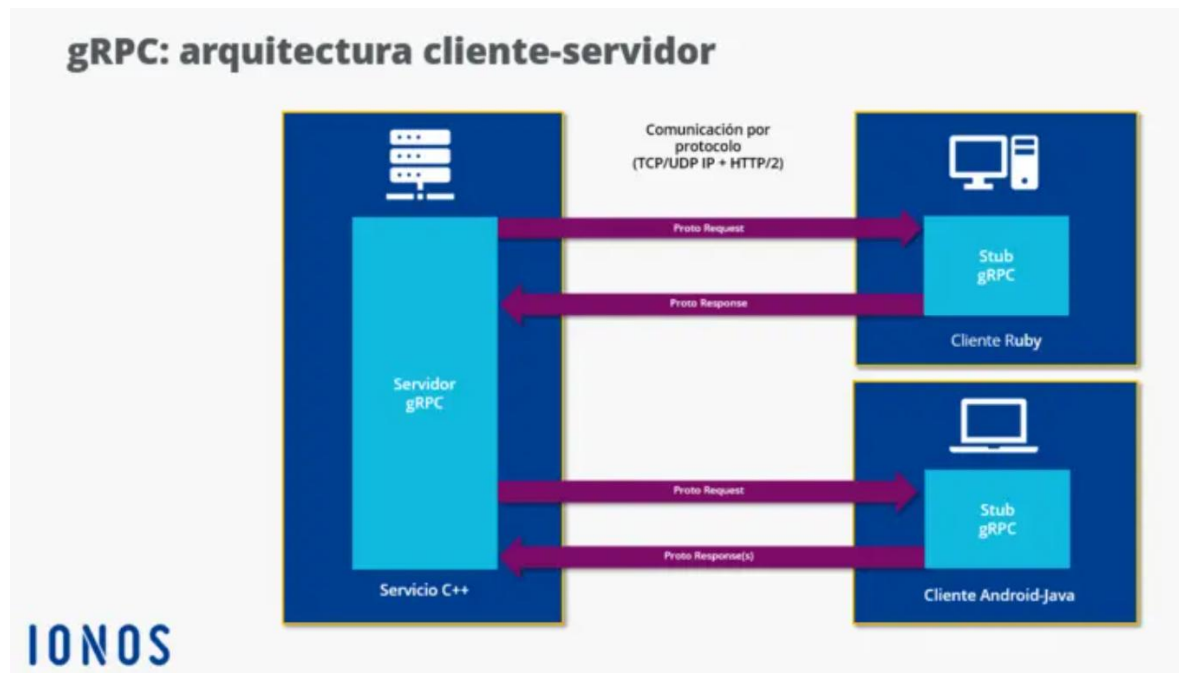


Ilustración 21 Arquitectura cliente servidor gRPC

gRPC se desarrolló para satisfacer esta necesidad, ofreciendo baja latencia y alto rendimiento a través de la serialización de datos y su uso del protocolo HTTP/2, capacidades de transmisión bidireccional, generación de código y más.

Por defecto, gRPC ejecuta el transporte de flujos de datos entre ordenadores alejados mediante HTTP/2 y gestiona la estructura y la distribución de los datos mediante los Protocol buffers desarrollados por Google. Estos últimos se guardan en forma de archivos de texto plano con la extensión “.proto”.

Google Protocol Buffers

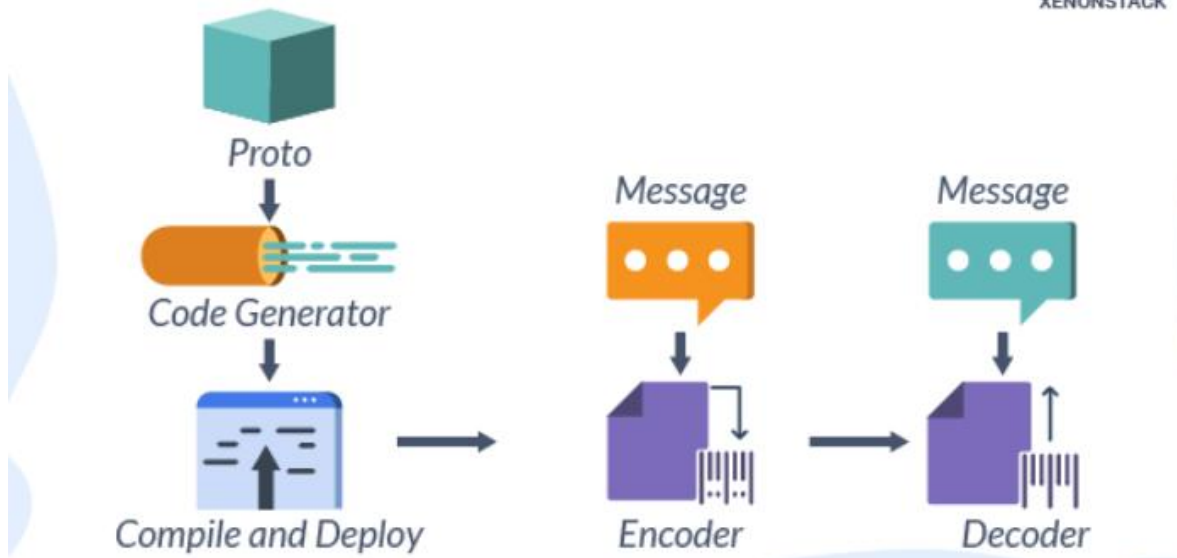


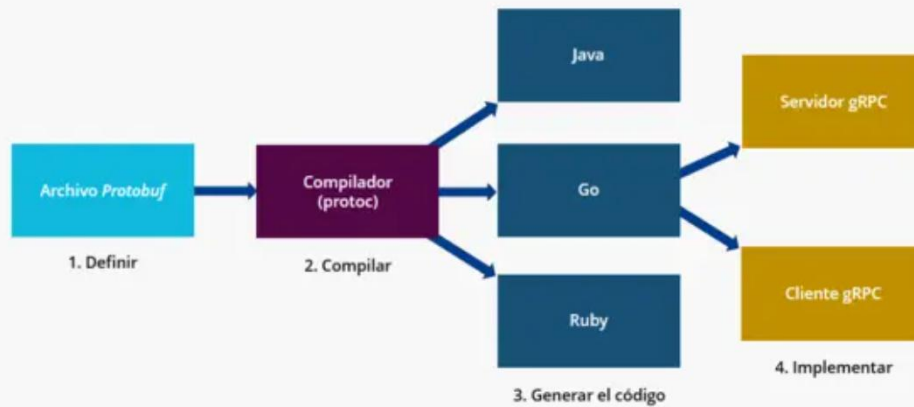
Ilustración 22 Archivos proto

Los Protocol Buffers (Protobuf) cumplen varias funciones en el sistema gRPC: sirven como lenguaje de descripción de interfaz (Interface Definition Language, IDL) y describen una interfaz de manera independiente de cualquier lenguaje, es decir, no están vinculados a un lenguaje de programación específico (p. ej., Java o C). También definen los servicios que se deben emplear y las funciones disponibles. Para cada función, puede indicarse qué parámetros se envían con una consulta y qué valor de respuesta se puede esperar.

Desde la perspectiva remota, los Protocol Buffers sirven como el **formato subyacente de intercambio de mensajes** que determina las estructuras, los tipos y los objetos de los mensajes. Son los elementos que hacen que el cliente y el servidor se “entiendan” y que funcionen como una unidad funcional y lo más eficiente posible, incluso a gran distancia.

El flujo de trabajo gRPC y primeros pasos con Protocol Buffers

Flujo de trabajo gRPC



IONOS

Ilustración 23 Flujo de trabajo gRPC

1. **Definición del contrato de servicio** para la comunicación entre procesos: se determinan los servicios que se deben aplicar y los parámetros y tipos de respuesta básicos a los que se puede acceder de manera remota.
2. **Generación del código gRPC** del archivo *.proto*: unos compiladores especiales (herramientas de líneas de comandos denominadas “protoc”) generan el código operativo con las clases correspondientes para un lenguaje de destino deseado (p. ej., C++, Java) a partir de los archivos *.proto* guardados.
3. **Implementación del servidor** en el lenguaje deseado.
4. **Creación del *stub* del cliente** que llama al servicio; a continuación, entre el servidor y el cliente se procesan las consultas.

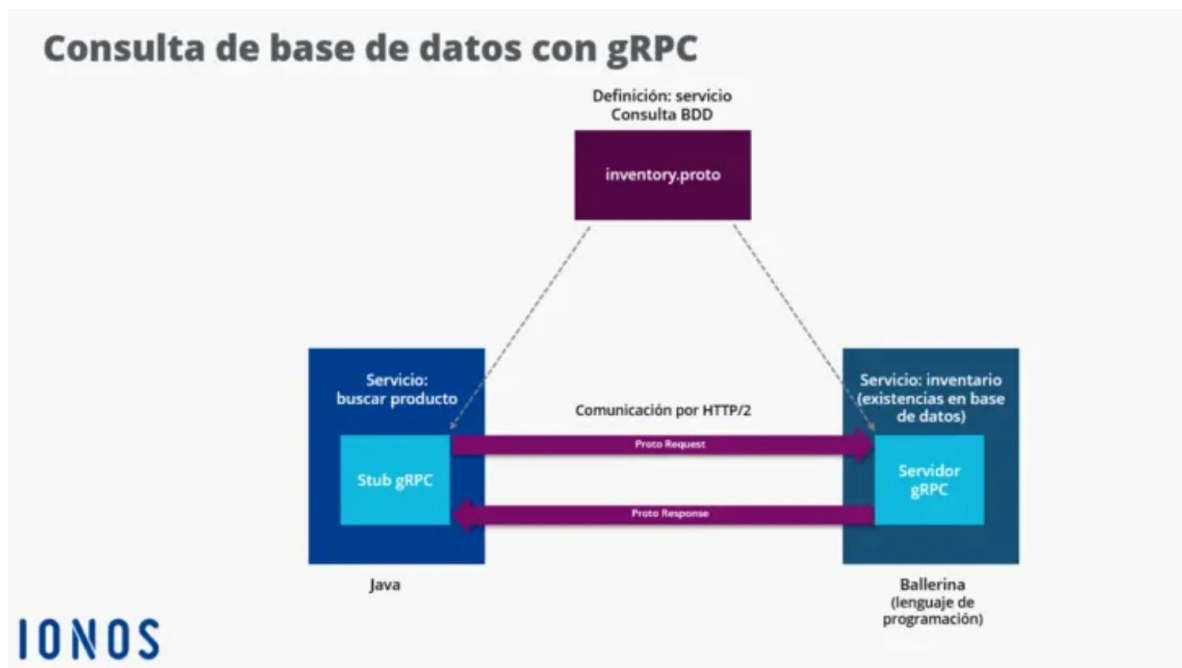


Ilustración 24 Consulta de base de datos con gRPC

Característica	gRPC	REST/JSON
Protocolo de transporte	HTTP/2 (multiplexado)	HTTP/1.1 (una petición por conexión)
Serialización	Protocol Buffers (binario)	JSON (texto)
Tamaño del mensaje	3-10x menor	Mayor
Tipado	Estricto (definido en .proto)	Dinámico (sin contrato obligatorio)
Generación de código	Automática en múltiples lenguajes	Manual o con OpenAPI
Streaming	Bidireccional nativo	Requiere WebSockets

Tabla 6 Ventajas de gRPC sobre REST en sistemas distribuidos

Definición de Contratos de Servicio (Archivos .proto)

El sistema utiliza un único archivo dbaas.proto que define todos los contratos entre microservicios. Esto garantiza coherencia en los tipos de datos intercambiados y evita incompatibilidades de versión.

Servicio gRPC	Microservicio	RPCs expuestos
GatewayService	ms-gateway (:50051)	Login, Register, Execute
AuthService	ms-auth (:50056)	Register, Login, ValidateToken
BrokerService	ms-broker (:50052)	Process
AdminService	ms-admin (:50053)	CreateDatabase, DropDatabase, ListDatabases, CreateTable, DropTable, ListTables
CrudService	ms-crud (:50054)	Insert, Find, Update, Delete
QueryService	ms-query (:50055)	Count, Sum, Avg, Distinct, Join

Tabla 7 Servicios definidos en dbaas.proto

Serialización de Datos y Tipos de Mensajes gRPC

El mensaje central del sistema es IROperation, la representación interna que viaja entre el broker y los servicios de ejecución. Todos los campos son de tipo string para máxima compatibilidad, con los campos complejos (fields, filter, data, options) serializados como JSON strings:

Campo	Tipo	Descripción
operation	string	"insert" "find" "update" "delete" "create_db" etc.

database	string	Nombre de la base de datos destino
table	string	Nombre de la tabla o colección
fields	string	JSON: ["campo1", "campo2"] o ["*"]
filter	string	JSON: {"edad": {"\$gt": 18}}
data	string	JSON: documento a insertar o campos a actualizar
options	string	JSON: parámetros extra (limit, join_table, on...)
role	string	Rol del usuario, reenviado para validación en cada MS
user_id	string	ID del usuario que originó la operación

Tabla 8 Mensajes tipo JSON

Documentación Detallada de Interfaces y Servicios

Catálogo de Servicios y Endpoints expuestos (API Gateway)

Una puerta de enlace de API es una capa de software que presenta un único punto de entrada para que los clientes (como aplicaciones web o móviles) accedan a múltiples servicios de backend, al tiempo que gestionan simultáneamente las interacciones cliente/servidor. Es un componente común en las arquitecturas de microservicios.

Las API ayudan a las empresas a utilizar servicios internos y de terceros sin tener que crear integraciones personalizadas para cada uno.

Las API gateways pueden servir tanto a usuarios internos que acceden a una aplicación propia o de terceros como a usuarios externos, por ejemplo clientes o asociados de negocios.

Una solicitud de datos de un cliente a una API se conoce como llamada a la API. La API gateway recibe una llamada API (a veces llamada solicitud API), la enruta a uno o más servicios de backend, recopila los datos solicitados y los entrega al cliente en una única respuesta combinada. De lo contrario, el cliente necesitaría interactuar

directamente con varias API para acceder a cada servicio o fuente de datos relevante.

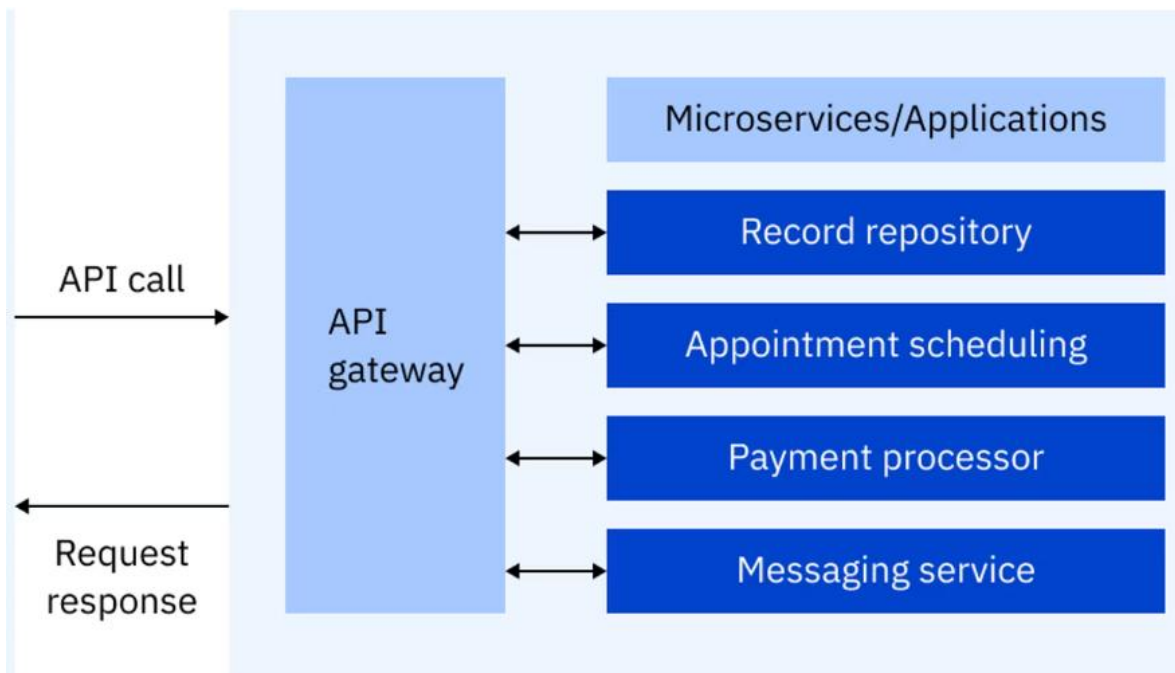


Ilustración 25 Funcionamiento API Gateway

En un entorno de microservicios, API Gateway a menudo manejan el tráfico norte-sur, enrutando las llamadas API de clientes externos al servicio de backend apropiado. A menudo trabajan junto con mallas de servicios, que manejan principalmente el tráfico este-oeste, o las comunicaciones entre servicios dentro del entorno de microservicios.

Hay algunas excepciones: se puede configurar una API gateway para enrutar el tráfico interno, especialmente en entornos modernos. Pero la puerta de enlace tiende a ubicarse en una capa arquitectónica separada, mientras que las mallas de servicios se despliegan junto con cada servicio o se integran con él para facilitar y gestionar las conexiones internas.

El API Gateway (ms-gateway) es el único servicio expuesto al cliente externo, escuchando en el puerto 50051. Expone tres operaciones RPC:

RPC	Request	Response	Descripción
Login	LoginRequest (username, password)	GatewayAuthResponse (success, token, role, message)	Autentica al usuario y retorna JWT + rol
Register	RegisterRequest (username, password, role)	GatewayAuthResponse (success, message)	Registra nuevo usuario en el sistema
Execute	ExecuteRequest (interface, payload, active_db, token)	ExecuteResponse (success, message, data)	Ejecuta cualquier operación SQL o NoSQL

Tabla 9 API Gateway

Campo interface en ExecuteRequest:

- "sql" — el campo payload contiene una query SQL como string.
- "nosql" — el campo payload contiene un objeto JSON serializado con la operación NoSQL.

Campo data en ExecuteResponse:

El campo data siempre es un JSON string con el resultado completo de la operación. Su estructura varía según la operación ejecutada: documentos para SELECT/JOIN, valor escalar para COUNT/SUM/AVG, lista de strings para DISTINCT, lista de bases de datos o tablas para SHOW.

Documentación Interna de Microservicios

A continuación, se documenta cada microservicio del sistema con sus operaciones RPC, mensajes de entrada y salida, y responsabilidades específicas. Todos los servicios internos son accesibles únicamente desde la red interna Docker; ningún cliente externo puede contactarlos directamente.

MS3 — Auth Service (puerto 50056)

Responsable de registro, autenticación y validación de tokens JWT. Es el único servicio que conoce las contraseñas de los usuarios (almacenadas con bcrypt). Solo el Gateway lo invoca directamente.

RPC	Request	Response	Descripción
Register	RegisterRequest (username, password, role)	AuthResponse (success, token="", message)	Crea usuario con contraseña hasheada en bcrypt. No emite token en el registro.
Login	LoginRequest (username, password)	AuthResponse (success, token=JWT, message)	Verifica credenciales con bcrypt y emite JWT firmado (HS256, 24h).
ValidateToken	TokenRequest (token)	TokenPayload (valid, user_id, username, role, message)	Decodifica y verifica el JWT. El Gateway lo llama en cada petición Execute.

Tabla 10 Auth Service MS3

MS4 — Admin Service (puerto 50053)

Gestiona la estructura lógica del sistema: creación y eliminación de bases de datos y tablas/colecciones en MySQL. Aplica segunda línea de defensa RBAC: todas las operaciones destructivas requieren rol admin.

RPC	Request	Response	Rol requerido
CreateDatabase	DatabaseRequest (database, role, user_id)	StatusResponse (success, message)	admin
DropDatabase	DatabaseRequest (database, role, user_id)	StatusResponse (success, message)	admin

ListDatabases	ListDbRequest (role, user_id)	ListDbResponse (success, databases[])	admin / write / read
CreateTable	CreateTableRequest (database, table, mode, schema, role, user_id)	StatusResponse (success, message)	admin
DropTable	TableRequest (database, table, role, user_id)	StatusResponse (success, message)	admin
ListTables	TableRequest (database, table="", role, user_id)	ListTablesResponse (success, tables[])	admin / write / read

Tabla 11 Admin Service MS4

MS5 — CRUD Service (puerto 50054)

Ejecuta operaciones de manipulación de datos. Detecta automáticamente si la tabla destino es relacional o una colección JSON (mediante `is_collection()`), y adapta las queries MySQL en consecuencia. Todos los RPCs reciben un `IROperation` y retornan `CrudResponse` o `FindResponse`.

RPC	Response	Rol requerido	Comportamiento
Insert	CrudResponse (success, message, affected_rows)	admin / write	Tabla: INSERT con columnas explícitas. Colección: INSERT con <code>_id</code> UUID y <code>_data</code> JSON.
Find	FindResponse (success, message, documents[])	admin / write / read	SELECT con WHERE dinámico. Colección: usa <code>JSON_EXTRACT</code> para filtrar dentro de <code>_data</code> .

Update	CrudResponse (success, message, affected_rows)	admin / write	Tabla: SET col=val. Colección: usa JSON_SET para modificar campos dentro de _data. Requiere filtro obligatorio.
Delete	CrudResponse (success, message, affected_rows)	admin / write	DELETE FROM tabla WHERE. Requiere filtro obligatorio para evitar borrado total accidental.

Tabla 12 CRUD Service MS5

MS6 — Query / Aggregation Service (puerto 50055)

Ejecuta consultas analíticas. Las agregaciones escalares (COUNT, SUM, AVG) usan cómputo paralelo con MPI. DISTINCT va directo a MySQL. JOIN soporta tablas normales (JOIN en MySQL) y colecciones (JOIN en Python con indexado por HashMap).

RPC	Response	Motor de cómputo	Descripción
Count	ScalarResponse (success, message, value)	MPI paralelo	Cuenta filas que cumplen el filtro. Distribuye lista de 1s entre workers MPI y suma parciales.
Sum	ScalarResponse (success, message, value)	MPI paralelo	Obtiene valores del campo desde MySQL y distribuye entre workers para suma parcial.
Avg	ScalarResponse (success, message, value)	MPI paralelo	Suma parciales de cada worker y divide entre total_count global para el promedio final.
Distinct	DistinctResponse (success, message, values[])	MySQL directo	SELECT DISTINCT campo. Colección: SELECT DISTINCT JSON_EXTRACT(_data, '\$campo').

Join	JoinResponse (success, message, documents[])	MySQL / Python	Tablas normales: INNER JOIN en MySQL. Colecciones: join en Python con índice HashMap O(n) sobre tabla_b.
------	--	----------------	---

Tabla 13 Query / Aggregation Service MS6

MS2 — Translation Broker (puerto 50052)

Recibe el payload crudo del Gateway, lo parsea con el parser correspondiente (sql_parser o nosql_parser), construye la IR y la enruta al microservicio destino (Admin, CRUD o Query) mediante router.py.

RPC	Request	Response	Descripción
Process	BrokerRequest (interface, payload, active_db, role, user_id)	BrokerResponse (success, message, data)	Parsea el payload, construye la IR e invoca el servicio destino según la operación detectada.

Tabla 14 Translation Broker MS2

Documentación de Colas y Formato de Mensajes (RabbitMQ)

Propiedad	Valor
Nombre de la cola	dbaas.audit
Tipo de cola	Durable (persiste reinicios)
Protocolo	AMQP 0-9-1
Puerto	5672 (AMQP) / 15672 (Management UI)
Productor	ms-gateway (después de cada Execute)

Consumidor	ms-audit
Delivery mode	2 (persistente en disco)
Prefetch count	1 (procesamiento secuencial)

Tabla 15 Propiedades de la cola de mensajes de RabbitMQ

Esquema JSON del payload de auditoría:

```
{ "user_id": "string (UUID)", "role": "admin|write|read", "operation": "string (nombre de operación)", "interface": "sql|nosql", "success": boolean }
```

Manejo de errores en la cola:

El sistema no implementa Dead Letter Queue (DLQ) explícita, pero usa NACK sin requeue para mensajes corruptos, enviándolos al dead letter exchange predeterminado de RabbitMQ. Si el consumidor no puede procesar un mensaje (JSON malformado, error de MySQL), lo rechaza sin reencolar para evitar loops infinitos de procesamiento fallido.

Infraestructura de Red y Despliegue Físico

Topología de Red de los Nodos Distribuidos

El sistema opera sobre una red local donde todos los microservicios se comunican mediante direcciones configurables a través de variables de entorno. La topología es de tipo estrella lógica con el API Gateway como punto central de entrada.

La **topología en estrella, o red en estrella**, es una configuración para una red de área local (LAN) en la que cada uno de los nodos están conectados a un punto de conexión central, tal como un concentrador, conmutador o una computadora. Esta topología es una de las configuraciones de red más usuales.

Por tanto, es una topología de red en la que cada parte individual de la red está conectada a un nodo central. La unión de estos dispositivos de la red al componente central se representa visualmente de forma similar a una estrella.

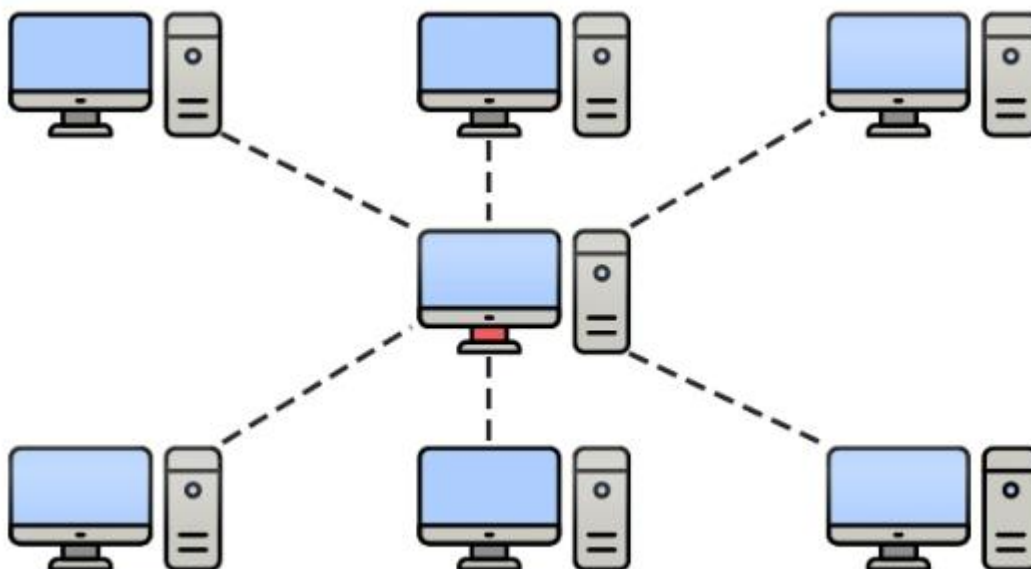


Ilustración 26 Topología de nodos en estrella

Todo el tráfico de datos procede del centro de la estrella. Así, este sitio central tiene el control de todos los nodos conectados a él. El concentrador central suele ser una computadora rápida e independiente y es responsable de enrutar todo el tráfico a los demás nodos.

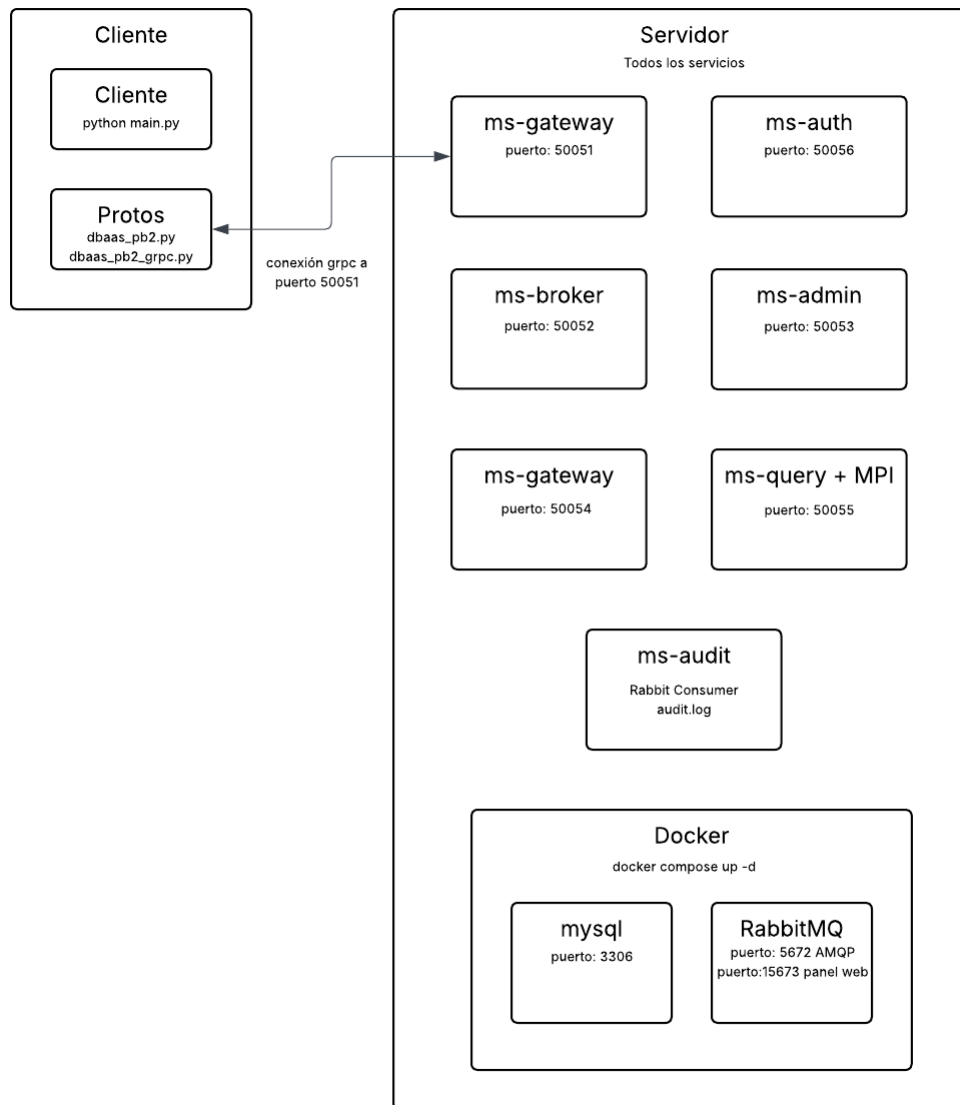
El nodo en el centro de la red trabaja como un servidor y los dispositivos periféricos ejercen como clientes.

Microservicio	Variable de entorno	Puerto por defecto	Protocolo
API Gateway	GATEWAY_ADDR / GATEWAY_PORT	50051	gRPC/HTTP2
Translation Broker	BROKER_ADDR / BROKER_PORT	50052	gRPC/HTTP2
Admin Service	ADMIN_ADDR / ADMIN_PORT	50053	gRPC/HTTP2
CRUD Service	CRUD_ADDR / CRUD_PORT	50054	gRPC/HTTP2

Query Service	QUERY_ADDR / QUERY_PORT	50055	gRPC/HTTP2
Auth Service	AUTH_ADDR / AUTH_PORT	50056	gRPC/HTTP2
MySQL	DB_HOST / DB_PORT	3306	MySQL protocol/TCP
RabbitMQ	RABBITMQ_HOST / RABBITMQ_PORT	5672 / 15672	AMQP / HTTP

Tabla 16 Propiedades de nodos

Diagrama de Despliegue UML



La infraestructura de despliegue del sistema está definida parcialmente por docker-compose.yml, que levanta los servicios de infraestructura base: MySQL 8.0 y RabbitMQ 3.13. Los seis microservicios Python se ejecutan como procesos independientes en el host, cada uno en su propio proceso Python.

Simulación de Escalabilidad y Clusterización (Filosofía de paso de mensajes/MPI adaptado a la infraestructura)

Las operaciones de agregación (COUNT, SUM, AVG) implementan computación paralela mediante MPI (Message Passing Interface), un estándar para comunicación entre procesos en sistemas de memoria distribuida (MPI Forum, 2021).

Arquitectura del cómputo paralelo:

1. aggregations.py obtiene los valores numéricos desde MySQL y los serializa como JSON.
2. Lanza `mpiexec -n 4 python mpi_worker.py` (4 procesos MPI por defecto, configurable).
3. El proceso rank 0 recibe los datos por stdin, los divide en chunks iguales y los distribuye.
4. Cada worker (rank 0..N-1) calcula su parcial local (sum, count, min, max).
5. `gather()` recolecta todos los parciales en rank 0, que combina el resultado final.
6. El resultado se imprime como JSON en stdout y aggregations.py lo captura.

Este patrón reproduce el modelo MapReduce: los workers realizan el map (cómputo local) y el rank 0 realiza el reduce (combinación de resultados). La complejidad del cómputo se distribuye en $O(n/p)$ por proceso, donde p es el número de procesos MPI.

Conclusiones

Evaluación del Cumplimiento de Objetivos (Transparencia e Independencia de Datos)

Objetivo	Estado	Evidencia en el código
Modularizar el sistema en microservicios	Cumplido	6 microservicios independientes con responsabilidades separadas
Abstraer la persistencia (transparencia)	Cumplido	IR unificada, detección automática de modo colección/tabla
Optimizar comunicación inter-servicio con gRPC	Cumplido	dbaas.proto define contratos estrictos, todos los MS usan gRPC
Asegurar el entorno con JWT y RBAC	Cumplido	ms-auth con bcrypt, role_guard.py con 3 roles y doble validación
Resiliencia asíncrona con RabbitMQ	Cumplido	Cola durable dbaas.audit con ACK manual y consumidor independiente
Computación distribuida con MPI	Cumplido	mpi_worker.py con scatter/gather para COUNT, SUM, AVG

Tabla 17 Tabla de cumplimiento

Limitaciones del Sistema DBaaS Desarrollado

- **Sin TLS en gRPC:** Todas las conexiones usan `grpc.insecure_channel()`. En un entorno de producción se requiere TLS mutuo para proteger los datos en tránsito.
- **Registro de admins sin restricción:** Cualquier usuario puede registrarse con rol admin sin autenticación previa. Se requiere un usuario administrador inicial con bootstrapping controlado.

- **Parser SQL limitado:** El parser basado en regex no soporta sintaxis SQL compleja (subconsultas, múltiples JOINS, GROUP BY, ORDER BY). Solo cubre el subconjunto definido en el proyecto.
- **Conexión RabbitMQ por evento:** El Gateway abre y cierra una conexión RabbitMQ por cada evento de auditoría, lo que es ineficiente.
- **MPI con datos en memoria:** el COUNT carga una lista de $1.0 * \text{total_filas}$ en RAM antes de enviarla a MPI. Con millones de filas esto puede causar OutOfMemoryError.

Referencias

IBM. (s. f.). ¿Qué es la base de datos como servicio (DBaaS)? IBM Think Topics. <https://www.ibm.com/es-es/think/topics/dbaas>

IONOS. (2024, 6 de mayo). ¿Qué es DBaaS (Database as a Service)? Digital Guide IONOS. <https://www.ionos.mx/digitalguide/servidores/known-how/que-es-dbaas/>

La Web del Programador. (s. f.). Manual práctico de SQL. <https://www.lawebdelprogramador.com/cursos/archivos/ManualPracticoSQL.pdf>

Universidad de Cantabria. (s. f.). Tema 1. NoSQL introducción [Diapositivas de PowerPoint]. OpenCourseWare. <https://ocw.unican.es/pluginfile.php/2444/course/section/2483/Tema%201.%20NoSQL%20introducci%C3%B3n.pdf>

acens. (2014, febrero). Bases de datos NoSQL: Qué son y cuándo elegir las [White paper]. Cloudhead. <https://www.acens.com/comunicacion/wp-content/images/2014/02/bbdd-nosql-wp-acens.pdf>

Instituto Nacional de Aprendizaje. (s. f.). Operaciones básicas de manipulación de datos. INA Virtual. https://www.inavirtual.ed.cr/pluginfile.php/2857420/mod_resource/content/1/MD_Recurso07_Operaciones%20b%C3%A1sicas%20de%20manipulacion%20de%20datos.pdf

LearnSQL.es. (2023, 20 de octubre). Funciones agregadas SQL: Una guía completa para principiantes. Blog de LearnSQL.es. <https://learnsql.es/blog/funciones-agregadas-sql-una-guia-completa-para-principiantes/>

GeneXurus Training. (s. f.). Control de acceso basado en roles (RBAC). <https://training.genexus.com/es/aprendiendo/pdf/control-de-acceso-basado-en-roles-rbac-pdf>

NetMentor. (2020, 11 de agosto). RabbitMQ y la comunicación asíncrona. <https://www.netmentor.es/entrada/rabbitmq-comunicacion-asincrona>

IBM. (s. f.). ¿Qué es gRPC? IBM Think Topics. <https://www.ibm.com/mx-es/think/topics/grpc>

IONOS. (2024, 13 de marzo). ¿Qué es gRPC? Explicación del protocolo de Google. Digital Guide IONOS. <https://www.ionos.es/digitalguide/servidores/known-how/que-es-grpc/>

IBM. (s. f.). ¿Qué es una API gateway? IBM Think Topics. <https://www.ibm.com/mx-es/think/topics/api-gateway>

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. Internet Engineering Task Force (IETF). <https://datatracker.ietf.org/doc/html/rfc7519>

NIST. (2017). Digital Identity Guidelines: Authentication and Lifecycle Management (SP 800-63B). National Institute of Standards and Technology. <https://pages.nist.gov/800-63-3/sp800-63b.html>

Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-Based Access Control Models. IEEE Computer, 29(2), 38–47. <https://doi.org/10.1109/2.485845>

Tanenbaum, A. S., & Van Steen, M. (2007). Distributed Systems: Principles and Paradigms (2nd ed.). Prentice Hall.

Google. (2024). gRPC — A high performance, open source universal RPC framework. <https://grpc.io/>

RabbitMQ. (2024). RabbitMQ Documentation. <https://www.rabbitmq.com/docs>

MPI Forum. (2021). MPI: A Message-Passing Interface Standard, Version 4.0. <https://www.mpi-forum.org/docs/mpi-4.0/mpi40-report.pdf>